

Cloud Run 배포 확인 및 최신 리비전 트래픽 적용 가이드

1. 메인 브랜치의 최신 코드 확인하기

먼저 **GitHub** 메인 브랜치에 최신 수정사항이 반영되었는지 확인합니다. 로컬 저장소가 있다면 PowerShell에서 아래 명령어를 실행하세요:

```
# Git 저장소 폴더로 이동 (예: C:\path\to\repo)
cd "C:\path\to\your\repository"

# 메인 브랜치로 체크아웃하고 최신 내용 가져오기
git checkout main
git pull origin main

# 마지막 커밋 및 index.html 수정 내용 확인
git log -1
git diff HEAD~1 HEAD -- path/to/index.html
```

- `git pull` 명령을 통해 **메인 브랜치가 원격 저장소의 최신 커밋과 동기화되었는지** 확인합니다.
- `git log -1` 은 현재 HEAD의 커밋 메시지와 해시를 보여줍니다. 이 커밋이 `index.html` 수정사항이 포함된 PR의 커밋인지 확인하세요.
- `git diff` 명령으로 마지막 커밋의 변경 내용을 확인하여 **`index.html`에 수정사항이 반영되었는지** 검증합니다.

⚠ **참고:** GitHub 웹사이트에서도 메인 브랜치의 최신 커밋과 파일 변경 내용을 확인할 수 있습니다. 해당 PR이 메인에 merge되었다면, 해당 커밋 해시와 내용을 기록해 둡니다.

2. Cloud Run에 최신 코드가 배포되었는지 확인하기

이제 **Cloud Run** 서비스에 최신 코드가 반영된 리비전(revision)이 배포되었는지 PowerShell에서 `gcloud` CLI로 확인해보겠습니다.

사전에 **gcloud CLI**가 설치 및 인증(`gcloud auth login`)되어 있어야 합니다. 또한 **프로젝트 ID**와 **Cloud Run 서비스 이름**, **리전(region)** 정보를 준비하세요.

1. **현재 활성화된 리비전 목록 조회:** Cloud Run 서비스의 리비전들을 나열하여 최근 배포 상태를 확인합니다.

```
$PROJECT = "<YOUR_GCP_PROJECT_ID>" # GCP 프로젝트 ID
$SERVICE = "<YOUR_CLOUD_RUN_SERVICE>" # Cloud Run 서비스 이름
$REGION = "<YOUR_CLOUD_RUN_REGION>" # 예: asia-northeast3 등

# Cloud Run 서비스의 리비전 목록 조회
gcloud run revisions list --service $SERVICE --region $REGION --project $PROJECT --platform managed
```

위 명령을 실행하면 해당 서비스의 모든 리비전이 출력됩니다. 예시 출력 형식은 다음과 같습니다 ¹ :

REVISION	ACTIVE	SERVICE	DEPLOYED	BY
✓ myservice-00017	yes	myservice	2025-10-21 05:12:34	user@example.com
myservice-00016		myservice	2025-10-20 09:45:10	user@example.com
...				

- **REVISION**: 리비전 이름 (예: myservice-00017). 숫자가 높을수록 최신 배포본입니다. - **ACTIVE**: 해당 리비전에 **트래픽이 할당되었는지** 나타냅니다. **yes** 이면 현재 서비스 트래픽의 일부 이상을 받고 있는 리비전입니다. - **DEPLOYED**: 해당 리비전의 배포 시각과 배포자입니다.

최신 리비전(myservice-00017 등)이 목록에 있는지 확인하고, **배포 시간과 배포자(email)**를 통해 GitHub Actions 배포가 제대로 반영되었는지 추적합니다. 배포자가 GitHub Actions에 사용된 서비스 계정이거나, 시간상이 PR 머지 이후인지를 확인하세요.

2. **서비스 상세 정보 확인**: Cloud Run 서비스의 상세 정보를 조회하여 현재 적용된 컨테이너 이미지와 리비전 정보를 확인합니다.

```
gcloud run services describe $SERVICE --region $REGION --project $PROJECT --platform managed
```

이 명령은 YAML 형식의 서비스 구성을 출력합니다. 여기서 확인할 내용: - **image**: 필드에 현재 배포된 컨테이너 이미지 경로가 나옵니다. 만약 GitHub Actions에서 이미지를 빌드해 배포했다면, 이미지 태그나 다이제스트에 커밋 해시 등이 포함되어 있을 수 있습니다. 해당 이미지 태그가 **최신 커밋의 해시나 버전을** 포함하고 있는지 확인하세요. - **environmentVariables** 나 **metadata.labels** 중에 **commit-sha** 라벨이 있는지 찾습니다. GitHub Actions로 배포했다면 Cloud Run 리비전에 **commit-sha** 라벨이 자동으로 붙어 있을 수 있습니다 ². 예를 들어:

```
metadata:
  labels:
    managed-by: github-actions
    commit-sha: 1a2b3c4d5e... # 배포된 커밋 해시
```

이 **commit-sha** 값이 **GitHub 메인 브랜치 최신 커밋과 일치**하는지 확인하면, Cloud Run에 배포된 컨테이너가 해당 커밋 기반임을 알 수 있습니다.

팁: 특정 필드만 보고 싶다면 **--format** 옵션을 사용할 수 있습니다. 예를 들어 현재 사용 중인 최신 리비전 이름만 확인하려면 `gcloud run services describe $SERVICE --format="value(status.latestReadyRevisionName)"` 명령을 사용할 수 있습니다 ³.

3. **최신 리비전의 index.html 확인 (선택 사항)**: 만약 Cloud Run에 배포된 컨테이너에 새로운 index.html이 포함되었는지 직접 확인하고 싶다면, Cloud Run 서비스 URL에서 index.html을 다운로드하여 확인할 수 있습니다. PowerShell에서 `Invoke-WebRequest`를 사용해 봅시다.

```
# Cloud Run 서비스 URL로부터 index.html 다운로드 (캐시 무효화 헤더 포함)
$url = "https://admin.standardai.co.kr/index.html"
Invoke-WebRequest $url -Headers @{"Cache-Control"="no-cache"} -OutFile
"deployed_index.html"
```

위 명령은 Cloud Run에 배포된 **실제 index.html 파일**을 `deployed_index.html`로 저장합니다. 파일을 열어보거나 (`notepad .\deployed_index.html`) 내용 중 수정한 부분이 들어있는지 검색해보세요.

- 만약 이 파일에 PR에서 수정한 내용이 있다면, **배포 자체는 성공한 것**입니다.
- 수정사항이 보이지 않는다면, 배포된 이미지에 변경사항이 포함되지 않았을 수 있으므로 다음 절차를 통해 문제를 해결해야 합니다.

3. 최신 리비전에 100% 트래픽 할당 여부 확인하기

Cloud Run은 기본적으로 새 리비전을 배포할 때 해당 리비전에 트래픽을 보낼지 여부를 설정할 수 있습니다. 만약 GitHub Actions에서 `no_traffic: true` 옵션이 사용되었거나 수동으로 트래픽 설정을 건너뛰었다면, 새 리비전이 배포되었어도 기존 리비전이 트래픽을 받고 있을 수 있습니다. 이를 확인하고자 **현재 트래픽이 어느 리비전에 할당되어 있는지** 점검합니다.

위 2단계의 **리비전 목록 출력 결과**에서 ACTIVE 열이 `yes`인 리비전이 **트래픽을 받고 있는 리비전**입니다. 일반적으로 100% 트래픽이라면 최신 리비전 하나만 ACTIVE여야 합니다. 만약 이전 리비전들도 `yes`로 표시된다면 트래픽이 분할되어 있거나 (`N%` 씩) 기존 리비전에 일부 남아있을 수 있습니다.

또는 서비스 설명 출력에서 `traffic:` 섹션을 찾아볼 수도 있습니다. `gcloud run services describe` 명령 출력에서 예시를 들면:

```
status:
traffic:
- type: TRAFFIC_TARGET_ALLOCATION_TYPE_LATEST
  revisionName: myservice-00017
  percent: 100
```

이처럼 **최신 리비전이 100%** 트래픽을 받도록 설정되어 있는지 확인하세요. Cloud Run 콘솔의 “트래픽 관리” 화면에서도 이를 시각적으로 확인할 수 있습니다. 아래 예시 이미지는 Cloud Run 콘솔에서 최신 리비전(`pinkfroid-00002`)이 **100% 트래픽을 받는 상태**를 보여줍니다 (이전 리비전은 0%) :

Cloud Run 콘솔의 리비전 및 트래픽 할당 예시 (최신 리비전에 100% 트래픽 할당)

- 만약 **최신 리비전에 100% 트래픽이 아니거나**, 여전히 이전 버전 UI가 노출된다면, 다음 단계를 따라 트래픽을 조정하거나 재배포가 필요합니다.

4. Cloud Run에 최신 리비전으로 트래픽 전환 또는 재배포하기 (필요 시)

위 확인 결과 새 리비전이 배포되었지만 트래픽을 받지 못하고 있거나, 아예 새 리비전이 없는 경우에는 수동으로 조치를 취합니다.

1) 최신 리비전에 트래픽 보내기: 새 리비전이 배포되었는데도 서비스가 구버전을 보여주는 경우, 트래픽이 이전 리비전에 남아있을 수 있습니다. 이때는 **gcloud CLI로 트래픽을 최신 리비전으로 일괄 전환**합니다. Cloud Run에서는 `gcloud run services update-traffic` 명령으로 트래픽 분배를 조정할 수 있습니다.

Cloud Run 서비스의 모든 트래픽을 최신 배포 리비전에 할당

```
gcloud run services update-traffic $SERVICE --to-latest --region $REGION --project $PROJECT --platform managed
```

위 명령은 현재 Cloud Run 서비스의 **최신 리비전에 100% 트래픽을 할당**합니다 ⁴. 실행 후 다시 `gcloud run services describe`로 traffic 섹션을 확인하여 percent 값이 최신 리비전에 100으로 된 것을 확인하세요. 또한 Cloud Run 웹 콘솔의 “트래픽” 설정에서도 변경사항이 반영됩니다.

i 참고: `--to-latest` 플래그는 이후 새로운 리비전이 배포될 때마다 자동으로 새 리비전에 트래픽을 넘기는 기능입니다 ⁵. 따라서 배포할 때 일일이 트래픽 옵션을 지정하지 않아도 항상 최신 버전이 노출됩니다. Canary 배포나 검증 후 트래픽 이전이 필요한 상황이 아니라면 이 옵션을 설정해두는 것이 편리합니다.

2) 최신 코드로 재배포: 만약 GitHub Actions가 최신 코드를 반영한 이미지를 배포하지 못했다면(예: 액션 실패나 누락), 수동으로 배포를 진행할 수 있습니다. 두 가지 방법이 있습니다.

- 이미 빌드된 이미지 재배포: GitHub Actions가 이미 Container Registry나 Artifact Registry에 이미지를 빌드해 두었다면, 해당 이미지를 가리켜 Cloud Run에 배포 가능합니다. 이미지 경로는 GitHub Action 로그 또는 Cloud Run 리비전 정보에서 확인한 `image:` 값을 사용합니다. 예를 들어:

```
# GitHub Actions가 빌드한 컨테이너 이미지로 Cloud Run에 배포 (이미지가 gcr.io 또는 artifact registry에 있다고 가정)
$IMAGE_URI = "gcr.io/your-project/your-service:commit-sha-tag"
gcloud run deploy $SERVICE --image $IMAGE_URI --region $REGION --project $PROJECT --platform managed --allow-unauthenticated
```

이 명령은 `$IMAGE_URI` 컨테이너로 새로운 리비전을 생성하고 배포합니다. 배포 후 자동으로 해당 리비전에 트래픽이 100% 할당되도록 하려면 `--traffic=latest=100` 옵션을 추가할 수도 있습니다.

- 소스 코드로부터 빌드/배포: 로컬에 최신 코드가 있고 Dockerfile이 구성되어 있다면, gcloud CLI가 직접 빌드하여 배포할 수 있습니다. 예를 들어 Dockerfile이 있는 경로에서:

```
# 현재 디렉터리의 소스로 Cloud Run 배포 (빌드 포함)
gcloud run deploy $SERVICE --source . --region $REGION --project $PROJECT --platform managed --allow-unauthenticated
```

이때 **.gcloudignore**와 **.gitignore** 설정에 유의하세요 (다음 섹션 참조). 빌드가 완료되면 Cloud Run에 바로 배포되며, 기본적으로 새 리비전에 트래픽 100%가 할당됩니다 (필요 시 `--no-traffic` 옵션으로 변경 가능).

배포 명령이 성공하면 출력에 새 리비전 이름과 URL 등이 표시됩니다. 이후 **웹 사이트를 새 리비전 URL로 열어 확인**하거나, 커스텀 도메인을 사용중이면 기존 도메인에서 새 내용이 보이는지 확인합니다.

5. 배포 누락 방지를 위한 .dockerignore / .gcloudignore 체크리스트

배포했는데도 `index.html` 수정사항이 반영되지 않는 경우, 빌드 컨텍스트에 해당 파일이 포함되지 않았을 수 있습니다. 이를 방지하기 위해 다음을 점검하세요:

- **.dockerignore 파일 확인:** Docker를 이용해 컨테이너를 빌드하는 경우, `.dockerignore`에 지정된 패턴의 파일들은 빌드 컨텍스트에서 제외됩니다 ⁶. `.dockerignore` 파일을 열어 `index.html`이나 해당 파일이 위치한 디렉터리가 제외되어 있지 않은지 확인하세요. 만약 불필요하게 제외되고 있다면 해당 항목을 삭제하거나 주석 처리합니다.
예를 들어 `.dockerignore`에 `public/`이나 `admin/` 폴더를 무작정 제외하고 있었다면, 그 폴더 내 `index.html`까지 빌드에 포함되지 않게 됩니다. 필요한 파일은 제외 패턴에서 빼주거나 `!filename` 식으로 포함시켜야 합니다.
- **.gcloudignore 파일 확인:** `gcloud run deploy --source` 등의 방식으로 소스에서 바로 배포할 때는 `.gcloudignore` 규칙이 적용됩니다. 기본적으로 `.gcloudignore`가 없으면 `.gitignore` 내용이 자동 적용되어, Git에서 무시한 파일은 배포에도 포함되지 않습니다 ⁷.
프로젝트 루트에 `.gcloudignore`가 있다면 열어보고, 없다면 `.gitignore` 내용을 확인하세요. `index.html`이나 **프론트엔드 빌드 산출물**이 `.gitignore`에 의해 무시되고 있지는 않은지 점검합니다. 만약 그런 파일이 있다면, `.gcloudignore`를 생성한 뒤 해당 파일을 포함(`!파일명` 문법)시키거나, `.gcloudignore` 상단에 `#!include:.gitignore` 줄을 제거하여 `.gitignore`를 무시하게 할 수도 있습니다 ⁷. 필요한 경우 `.gcloudignore`를 적절히 편집하여 배포 시 원하는 파일이 제외되지 않도록 합니다.
- **빌드 출력 확인:** 배포 과정에서 해당 파일이 이미지에 포함되었는지 확인하려면, Dockerfile에 일시적으로 디버그 명령을 넣어보는 방법도 있습니다. 예를 들어 Dockerfile의 마지막에 `RUN ls -R /path/to/your/app`를 넣고 이미지를 빌드하면, 빌드 로그에 포함된 파일 목록이 출력됩니다. 이를 통해 `index.html`이 실제로 이미지에 들어갔는지 확인할 수 있습니다.

이러한 체크리스트를 통해 배포 전에 파일 누락 문제를 조기에 발견하고 수정할 수 있습니다.

6. 배포 후 브라우저 캐시 무시하고 최신 UI 확인하기

Cloud Run 배포가 완료되고 최신 리비전에 트래픽도 전환되었다면, 브라우저에 캐시된 오래된 자원이 문제일 수 있습니다. 최종 사용자 측 브라우저가 이전 버전의 정적 파일 (특히 JavaScript, CSS)을 캐시하고 있어 새 `index.html`에서 요청하는 최신 파일을 받아오지 못하는 경우가 있습니다. 이를 해결하려면:

- **브라우저 캐시 비우기/강력 새로고침:** Chrome 기준으로 웹 페이지에서 `Ctrl + Shift + R` (맥은 `Cmd + Shift + R`)을 눌러 캐시를 무시한 새로고침을 수행합니다. 또는 개발자 도구(DevTools)를 열고 **Network 패널**에서 "Disable cache" 옵션을 체크한 뒤 새로고침해도 됩니다. 이렇게 하면 브라우저가 서버에서 최신 파일들을 다시 받아옵니다.
- **시크릿 모드/인콰이어토 모드 사용:** 시크릿 창을 열고 사이트에 접속하면 브라우저 캐시 영향을 받지 않고 최신 콘텐츠를 불러올 수 있습니다.
- **URL 파라미터 활용:** 정적 파일 요청에 쿼리스트링을 임시로 추가해도 캐시를 회피할 수 있습니다. 예를 들어 `https://admin.standardai.co.kr/index.html?refresh=123`처럼 임의의 파라미터를 붙여 요청하면 새로운 요청으로 인식됩니다. 다만 이 방법은 일시적 확인용으로 사용하세요.

위 방법으로 강제로 최신 HTML/JS를 불러왔을 때 수정된 UI가 나타나는지 확인합니다. 만약 이렇게 해도 변화가 없다면, 여전히 새로운 빌드가 반영되지 않은 것이므로 앞서 언급한 리비전 상태 및 빌드 구성을 다시 점검해야 합니다.

7. (선택 사항) PowerShell 자동화 스크립트 예시

아래는 위의 검증 절차 중 일부 명령들을 하나의 PowerShell 스크립트로 묶은 예시입니다. 이 스크립트는 메인 브랜치 동기화부터 Cloud Run 서비스의 최신 리비전 및 트래픽 상태를 확인하고, 필요 시 최신 리비전에 트래픽을 몰아주는 것까지 포함합니다. 사용자 환경에 맞게 변수(\$PROJECT, \$SERVICE, \$REGION 등)를 수정한 후 사용하세요:

```
# 설정: 사용자별 변수
$RepoPath = "C:\path\to\your\repository" # 로컬 깃 저장소 경로
$PROJECT = "<YOUR_GCP_PROJECT_ID>" # GCP 프로젝트 ID
$SERVICE = "<YOUR_CLOUD_RUN_SERVICE>" # Cloud Run 서비스 이름
$REGION = "<YOUR_CLOUD_RUN_REGION>" # Cloud Run 서비스 리전 (예: asia-northeast3)

# 1. Git 메인 브랜치 최신화
Write-Host "`n[1] Git 메인 브랜치 최신화 중..."
Set-Location $RepoPath
git checkout main
git pull origin main

# 2. 최신 커밋 정보 확인
$latestCommit = git log -1 --pretty=format:"%H - %s"
Write-Host "현재 HEAD 커밋: $latestCommit"

# 3. Cloud Run 서비스 리비전 목록 확인
Write-Host "`n[2] Cloud Run 리비전 목록 확인 중..."
$revisions = gcloud run revisions list --service $SERVICE --region $REGION --project $PROJECT --platform managed
Write-Host $revisions

# 4. 최신 리비전에 트래픽 100% 할당 여부 검사
Write-Host "`n[3] 최신 리비전 트래픽 할당 여부 검사..."
$serviceDesc = gcloud run services describe $SERVICE --region $REGION --project $PROJECT --platform managed --format="value(status.traffic)"
Write-Host "트래픽 설정: $serviceDesc"
if ($serviceDesc -notlike "**percent=100*") {
    Write-Warning "경고: 최신 리비전에 100% 트래픽이 할당되지 않았습니다. 트래픽을 최신 리비전으로 변경합니다."
    gcloud run services update-traffic $SERVICE --to-latest --region $REGION --project $PROJECT --platform managed
} else {
    Write-Host "최신 리비전이 이미 100% 트래픽을 받고 있습니다."
}

# 5. 웹 요청으로 index.html 내용 확인
Write-Host "`n[4] 배포된 index.html 내용 가져오기..."
$resp = Invoke-WebRequest "https://admin.standardai.co.kr/index.html" -Headers @{ "Cache-Control"="no-cache" }
if ($resp.StatusCode -eq 200) {
```

```

$contentSnippet = $resp.Content.Substring(0, [Math]::Min(500, $resp.Content.Length))
Write-Host "index.html 응답 (처음 500자):"
Write-Host $contentSnippet
} else {
    Write-Error "사이트에서 index.html을 가져오지 못했습니다. HTTP 상태 코드: $($resp.StatusCode)"
}

```

위 스크립트를 실행하면 각 단계별로 진행사항과 중요한 출력이 나타납니다. 이로써 **메인 브랜치 최신화 → Cloud Run 리비전/트래픽 확인 → 필요 시 트래픽 갱신 → 실제 서비스의 index.html 일부 표시**까지 한 번에 수행할 수 있습니다.

스크립트 출력 예시:

```

[1] Git 메인 브랜치 최신화 중...
Already on 'main'
Your branch is up to date with 'origin/main'.

현재 HEAD 커밋: 1a2b3c4d5e - Fix UI issue on admin index.html

[2] Cloud Run 리비전 목록 확인 중...
REVISION      ACTIVE  SERVICE  DEPLOYED          BY
✓ admin-00012  yes    admin    2025-10-21 09:15:42 [email protected]
  admin-00011          admin    2025-10-20 14:03:27 [email protected]
... (생략) ...

[3] 최신 리비전 트래픽 할당 여부 검사...
트래픽 설정: [{'latestRevision': True, 'percent': 100}]
최신 리비전이 이미 100% 트래픽을 받고 있습니다.

[4] 배포된 index.html 내용 가져오기...
index.html 응답 (처음 500자):
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title>Standard AI Admin - v2.3</title>
<!-- ... -->

```

위와 같이 정상적인 경우 최신 index.html의 일부 내용(예: 변경된 버전 표시 v2.3 등)이 출력됩니다. 만약 오류가 있다면 스크립트의 경고/오류 메시지를 확인하여 앞선 단계들을 다시 점검하세요.

이상의 절차를 따라가면 **GitHub PR로 변경된 index.html이 Cloud Run에 올바르게 배포되고, 최신 리비전에 트래픽이 적용되어 사용자에게 새 UI가 제공**되는지 확인할 수 있습니다. 또한 문제 발생 시 어느 단계에서 이슈가 있는지 파악하고 적절한 대응(재배포 또는 설정 수정 등)을 할 수 있을 것입니다. **배포 후에는 브라우저 캐시를 비우거나 강력 새로고침하여 최신 변경사항을 확인**하는 것도 잊지 마세요. 필요한 경우 위 PowerShell 명령어들을 응용하여 자동화된 배포 검증 스크립트를 구성하면 편리하게 반복 확인을 수행할 수 있습니다.

참고 자료: Cloud Run 리비전 관리 방법 ⁸ ¹ , Cloud Run 트래픽 할당 가이드 ⁴ ⁵ , Cloud Build 배포 시 파일 제외 규칙 ⁶ ⁷ .

¹ **cdn - How to check if the latest Cloud Run revision is ready to serve - Stack Overflow**

<https://stackoverflow.com/questions/61657875/how-to-check-if-the-latest-cloud-run-revision-is-ready-to-serve>

² **GitHub - google-github-actions/deploy-cloudrun: A GitHub Action for deploying services to Google Cloud Run.**

<https://github.com/google-github-actions/deploy-cloudrun>

³ **Gcloud Run - Find latest revision's name - Stack Overflow**

<https://stackoverflow.com/questions/68955551/gcloud-run-find-latest-revisions-name>

⁴ **Rollbacks, gradual rollouts, and traffic migration | Cloud Run | Google Cloud**

<https://cloud.google.com/run/docs/rollouts-rollbacks-traffic-migration>

⁵ **how can I ensure traffic is being served only on my new cloud run service deployment? - Stack Overflow**

<https://stackoverflow.com/questions/62566620/how-can-i-ensure-traffic-is-being-served-only-on-my-new-cloud-run-service-deploy>

⁶ ⁷ **docker - `gcloud builds submit` for Cloud Run - Stack Overflow**

<https://stackoverflow.com/questions/55846611/gcloud-builds-submit-for-cloud-run>

⁸ **Manage revisions | Cloud Run | Google Cloud**

<https://cloud.google.com/run/docs/managing/revisions>